

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-22 (b1e550e+수정중) → 2026-08-24 · 868c402+수정중 | 2개 파일 · +3 / -2 줄 · 코드 16쪽 | 새 코드북 79쪽

바뀐 내용

- 점유대역폭 재기가 신호 끝나고 한참 떨어진 외톨이 선(믹싱된 DC 누설)까지 대역으로 세던 결함을 고쳤다. 이것 때문에 참 baud 1296 이 '대역과 안 맞다'고 기각되고 엉뚱한 2744 가 이겼다 — 고친 뒤 같은 캡처를 다시 분석하면 qpsk bd1296 이 나와야 한다 (lock 은 낮아도 정상).
- 리포트 JSON 에 occupied(성상도에서 실제로 점유된 이상점 개수)를 넣었다. 이미 손으로 추가했다면 pipeline.py 의 초록 줄 한 줄 과 같은 모양인지 대조만 하면 된다.

□ <code>signus/dsp.py</code>	+2	-1	2쪽	코드북 15쪽
□ <code>signus/pipeline.py</code>	+1	-1	8쪽	코드북 36쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/dsp.py +2 -1 · 코드북 15쪽 · 새로 쓸 줄 302-303

```

1 1 """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2 2 anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3 3
4 4 from fractions import Fraction
5 5
6 6 import numpy as np
7 7 from scipy.ndimage import uniform_filter1d
8 8 from scipy.signal import get_window, hilbert, resample_poly, stft, welch
9 9
10 10 from ._accel import dd_carrier
11 11 from .constellations import ideal_points
12 12
13 13
14 14 def analytic(x: np.ndarray) -> np.ndarray:
15 15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16 16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17 17     overflows on the  $|x|^2$  the estimators take)."""
18 18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19 19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows  $|x|^2$  is corrupt t
20 20     oo
21 21     if np.iscomplexobj(x):
22 22         return x.astype(np.complex128)
23 23     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
24 24
25 25 def _parab(ydb: np.ndarray, k: int) -> float:
26 26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27 27     if k <= 0 or k >= ydb.size - 1:
28 28         return 0.0
29 29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30 30     d = a - 2 * b + c
31 31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32 32     # (d~0) or non-max triple can explode the ratio and drive baud negative -> a resample crash
33 33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34 34
35 35
36 36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37 37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38 38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39 39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40 40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41 41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42 42     lab = np.zeros(a.size, dtype=int)
43 43     for _ in range(iters):
44 44         d = np.abs(a - c[0])
45 45         lab = np.zeros(a.size, dtype=int)
46 46         for j in range(1, k):
47 47             dj = np.abs(a - c[j])
48 48             closer = dj < d
49 49             lab[closer] = j
50 50             d[closer] = dj[closer]
51 51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52 52         if np.allclose(nc, c):
53 53             break
54 54         c = nc
55 55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56 56
57 57

```

```

58 58 # --- burst detection -----
59 59
60 60 _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, no
   61 t a
61 61 # burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/Q
   62 AM
62 62 # peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared o
   63 ver
63 63 # a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with marg
   64 in.
64 64
65 65
66 66 def _otsu(v: np.ndarray, bins: int = 128) -> float:
67 67     """Otsu split threshold of a 1-D value distribution (log powers, column scores)."""
68 68     hist, edges = np.histogram(v, bins=bins)
69 69     centers = (edges[:-1] + edges[1:]) / 2
70 70     w = np.cumsum(hist).astype(float)
71 71     m = np.cumsum(hist * centers)
72 72     mb = m / np.where(w > 0, w, 1)
73 73     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
74 74     return float(centers[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
75 75
76 76
77 77 def _cell_power(x: np.ndarray, fs: float, nperseg: int = 256) -> tuple[np.ndarray, int, int]:
78 78     """Waterfall cell power for detection: (P[fbin, col], hop, nperseg). nperseg
79 79     auto-shrinks on short records; shared core for find_bursts (and, later, survey)."""
80 80     nperseg = int(min(nperseg, max(64, 1 << int(np.log2(max(x.size // 2, 64)))))
81 81     hop = nperseg // 2
82 82     win = get_window("blackmanharris", nperseg)
83 83     _, _, z = stft(x, fs=fs, window=win, nperseg=nperseg, noverlap=nperseg - hop,
84 84     return_onesided=False, boundary=None, padded=False)
85 85     return np.abs(z) ** 2, hop, nperseg
86 86
87 87
88 88 def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
89 89     """Waterfall (cell-level) burst detector. Per-bin noise floors give the same
90 90     narrowband processing gain the operator's eye gets on a spectrogram (a burst at
91 91     wideband 0 dB is +12 dB per cell when it occupies 1/16 of the band); column
92 92     scores then drive the run/merge/guard machinery on the time axis.
93 93     Returns bursts in time order, or [(0, size)] when nothing stands out."""
94 94     n = x.size
95 95     pw = np.abs(x) ** 2
96 96     # A burst is a TIME-ENERGY event. A continuous signal that merely shuffles its spectrum
97 97     # (multi-tone FSK) lights different cells per column and once shattered into 78 phantom
98 98     # bursts -- but its wideband envelope is FLAT. Envelope Otsu separation measured 0.014-
99 99     # 0.052 decades on continuous FSK vs >=0.30 on every real burst regime: nothing turns
100 100     # on or off -> the whole record is the burst.
101 101     lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
102 102     e_thr = _otsu(lp)
103 103     e_lo, e_hi = lp[lp < e_thr], lp[lp >= e_thr]
104 104     if not e_lo.size or not e_hi.size or float(e_hi.mean() - e_lo.mean()) < 0.12:
105 105         return [(0, n)]
106 106     # nperseg 128: bin 7.8 kHz keeps the narrowband gain for real signal widths while the
107 107     # 64-sample hop resolves inter-burst gaps down to ~200 samples (256 could not split a
108 108     # 300-sample gap -- no column fits inside it).
109 109     P, hop, nperseg = _cell_power(x, fs, nperseg=128)
110 110     if P.shape[1] < 4: # too few columns to tell bursts from record
111 111         return [(0, n)]
112 112     # Per-bin floor at a LOW percentile: tracks coloured noise bin by bin and stays on the
113 113     # noise even when a bin is signal-occupied up to ~90% of the time (high-duty trains).

```

```

114 114 # A record-filling signal owns its bins' floors entirely -> flat score -> [(0, n)].
115 115 floor_f = np.percentile(P, 10, axis=1)[: , None]
116 116 r = P / (floor_f + 1e-30)
117 117 k = max(3, P.shape[0] // 16)
118 118 sc = np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30) # column score (decades)
119 119 # A single noise column can spike +0.53 decades over base -- OVERLAPPING the weakest
120 120 # real bursts (+0.57). Like the eye, the discriminator is PERSISTENCE, not height: a
121 121 # 3-column smooth drops the noise worst-case to 0.31-0.33 while a real burst (>=3
122 122 # columns) keeps its level (0.56+) -- the gap the fixed bars below live in.
123 123 sc = uniform_filter1d(sc, 3)
124 124 # The noise base is the LOWEST MODE of the score histogram (Otsu, as the wideband
125 125 # version proved out): fixed quantile anchors cannot serve every duty regime, and
126 126 # spread ESTIMATES broke three ways (median blind >50% duty, two-sided MAD inflated by
127 127 # skirts, one-sided by the score's left tail). The margins are FIXED instead: a noise
128 128 # column's score is distribution-pinned at C = log10((ln(nbins/k)+1)/0.105) whatever
129 129 # the noise level, so its fluctuation is a constant of (nbins, k) -- measured worst
130 130 # 0.33 over 24 noise records vs 0.56 for the weakest keeper burst.
131 131 thr = _otsu(sc, bins=64)
132 132 quiet = sc[sc < thr]
133 133 if not quiet.size:
134 134     return [(0, n)]
135 135 base = float(np.median(quiet))
136 136 # No noise-quiet column at all (base far above the pinned C): a continuous signal
137 137 # shuffling its spectrum (a 4-tone FSK read as 78 phantom bursts before this guard).
138 138 c_noise = float(np.log10((np.log(P.shape[0] / k) + 1) / 0.105))
139 139 if base > c_noise + 0.25:
140 140     return [(0, n)]
141 141 hi = base + 0.45
142 142 lo = base + 0.25
143 143 above_lo = sc >= lo
144 144 dif = np.diff(above_lo.astype(np.int8))
145 145 starts = list(np.where(dif == 1)[0] + 1)
146 146 ends = list(np.where(dif == -1)[0] + 1)
147 147 if above_lo[0]:
148 148     starts.insert(0, 0)
149 149 if above_lo[-1]:
150 150     ends.append(sc.size)
151 151 runs = [(s, e) for s, e in zip(starts, ends, strict=True) if (sc[s:e] >= hi).any()]
152 152 if not runs:
153 153     return [(0, n)]
154 154 merged = [list(runs[0])]
155 155 for s, e in runs[1:]:
156 156     # A real inter-burst gap returns to the score base; a marginal-level burst dips just
157 157     # below lo without reaching it and would otherwise shatter into fragments. Merge
158 158     # across any valley whose median stays off the base.
159 159     valley = sc[merged[-1][1]:s]
160 160     if s - merged[-1][1] < 2 or float(np.median(valley)) > base + 0.12:
161 161         merged[-1][1] = e
162 162     else:
163 163         merged.append([s, e])
164 164 # Shape gate: a modulated burst has a near-flat raw envelope (peak/mean ~ a few); an
165 165 # impulse smear is one spike over noise (ratio ~span). Reject spike-dominated candidates.
166 166 out, spiked = [], 0
167 167 for c0, c1 in merged:
168 168     if c1 - c0 < 2: # single-column flicker (noise / impulse smear)
169 169         continue
170 170     s, e = max(0, c0 * hop), min(n, (c1 - 1) * hop + nperseg)
171 171     if c1 - c0 >= 6:
172 172         # long bursts: trim one hop of smear/quantization slack per edge. The noise
173 173         # tail otherwise fed to the demod sits on a knife edge for dense QAM (58 extra

```

```

174 174         # leading samples flipped a 16qam@24dB slice to 64qam lock 18). Short runs keep
175 175         # their full span -- trimming there would eat into the burst itself.
176 176         s, e = s + hop, e - hop
177 177         seg = pw[s:e]
178 178         if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
179 179             spiked += e - s
180 180             continue
181 181         out.append((int(s), int(e)))
182 182         # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
183 183         # inside the strongest burst killed its candidate, returning the survivors would hand
184 184         # analyze() the wrong emitter -- fall back instead. Only spike kills count: the
185 185         # single-column drops above are routine flicker filtering.
186 186         covered = sum(e - s for s, e in out)
187 187         if out and covered < 0.6 * (covered + spiked):
188 188             return [(0, n)]
189 189         # A continuous signal OWNS its bins' floors and is invisible to the cell path -- but its
190 190         # edge transients are not, and once returned [(0, 192)] for a 64k channel whose 16qam
191 191         # ran the whole record. If the detections cover almost none of the envelope's high-power
192 192         # mass, the cell path missed the main event: return the ENVELOPE's high span instead
193 193         # (not the raw record -- a dead tail fed to the demod flipped that 16qam to 64qam).
194 194         # 0.1 keeps partial detections (a strong burst next to a lo-only sibling covers ~30%).
195 195         if out and covered < 0.1 * int((lp >= e_thr).sum()):
196 196             idx = np.where(lp >= e_thr)[0]
197 197             return [(int(idx[0]), int(idx[-1]) + 1)]
198 198         return out or [(0, n)]
199 199
200 200
201 201 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
202 202     """The most energetic burst of find_bursts."""
203 203     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
204 204
205 205
206 206 # --- carrier estimation -----
207 207
208 208 def est_carrier(
209 209     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
210 210     nfft: int = 65536, blocks: int = 4,
211 211 ) -> tuple[float, int, bool]:
212 212     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
213 213     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
214 214     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
215 215     best_par, best_s, sym = -1.0, None, powers[0]
216 216     for p in powers:
217 217         xp = x ** p
218 218         blk = xp.size // blocks
219 219         segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
220 220         acc = np.zeros(nfft)
221 221         for s in segs:
222 222             acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
223 223             spec = acc / len(segs)
224 224             par = spec.max() / spec.mean()
225 225             if par > best_par:
226 226                 best_par, best_s, sym = par, spec, p
227 227
228 228     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
229 229         return 0.0, int(powers[0]), True
230 230     ydb = 20 * np.log10(best_s + 1e-20)
231 231     k = int(np.argmax(best_s))
232 232     f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
233 233     fc = f_peak / sym

```

```

234 234     ambiguous = abs(sym * fc) > 0.4 * fs
235 235     return float(fc), int(sym), bool(ambiguous)
236 236
237 237
238 238 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
239 239     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
240 240     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
241 241     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
242 242     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
243 243     f, pxx = Welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
244 244     p = np.maximum(pxx - np.median(pxx), 0)
245 245     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
246 246     # 8psk beyond 0.3125*fs with no candidate).
247 247     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
248 248     if abs(fc + k * fs / sym) < 0.5 * fs]
249 249     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
250 250     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum
    l, l,    ()))
251 251
252 252
253 253 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
254 254     """Downconvert by fc (complex heterodyne)."""
255 255     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
256 256
257 257
258 258 # --- symbol rate + rolloff -----
259 259
260 260 def est_baud(
261 261     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
262 262     blocks: int = 4,
263 263 ) -> tuple[float, float]:
264 264     """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
265 265     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
266 266     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
267 267     u = np.abs(x) ** 2
268 268     u = u - u.mean()
269 269     blk = u.size // blocks
270 270     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
271 271     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
272 272     acc = np.zeros(nfft // 2 + 1)
273 273     for s in segs:
274 274         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
275 275     spec = acc / len(segs)
276 276     freqs = np.fft.rfftfreq(nfft, 1 / fs)
277 277
278 278     lo = lo if lo is not None else 0.005 * fs
279 279     hi = hi if hi is not None else 0.45 * fs
280 280     band = (freqs >= lo) & (freqs <= hi)
281 281     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
282 282         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
283 283     idx = np.where(band)[0]
284 284     k = idx[int(np.argmax(spec[idx]))]
285 285     ydb = 20 * np.log10(spec + 1e-20)
286 286     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
287 287     conf = spec[k] / (np.median(spec[band]) + 1e-20)
288 288     return float(baud), float(conf)
289 289
290 290
291 291 def occupied_bw(x: np.ndarray, fs: float) -> float:
292 292     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way

```

```

293 293         from the 25th-percentile floor to the 90th-percentile in-band level, walk
294 294         outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
295 295         Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
296 296         f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
297 297         floor = np.percentile(pxx, 25)
298 298         order = np.argsort(np.abs(f))
299 299         idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
300 300         if idx.size == 0:
301 301             return 0.0
302 +         gaps = np.flatnonzero(np.diff(idx) > 20) # 틈 정지가 없으면 대역 밖 스퍼(믹싱돼
303 +         last = idx[gaps[0]] if gaps.size else idx[-1] # -fc 에 떨어진 LO 누설)가 bw 를 부풀린다
304 304         return float(2 * np.abs(f[order][last]))
305 306
306 307 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
307 308     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
308 309     n = x.size
309 310     nper = min(8192, max(256, n // 8))
310 311     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
311 312     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
312 313     af = np.abs(f)
313 314     inband = np.median(pxx[af < 0.25 * baud])
314 315     noise = np.median(pxx[af > 0.9 * baud])
315 316     thr = noise + 0.10 * (inband - noise)
316 317     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
317 318     if not band.any():
318 319         return 0.35
319 320     raw = 2 * af[band].max() / baud - 1
320 321     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
321 322
322 323 # --- resampling + matched filter -----
323 324
324 325
325 326 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
326 327     """Rational resample to exactly sps samples/symbol."""
327 328     r = Fraction(sps * baud / fs).limit_denominator(2000)
328 329     return resample_poly(x, r.numerator, r.denominator)
329 330
329 331
331 332 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
332 333     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
333 334     |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
334 335     m = (span * sps) // 2
335 336     t = np.arange(-m, m + 1) / sps
336 337     a = alpha
337 338     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
338 339     den = np.pi * t * (1 - (4 * a * t) ** 2)
339 340     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
340 341     h[t == 0] = 1 - a + 4 * a / np.pi
341 342     if a > 0:
342 343         s = np.isclose(np.abs(t), 1 / (4 * a))
343 344         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
344 345             + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
345 346     return h / np.sqrt(np.sum(h ** 2))
346 347
347 348
348 349 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
349 350     """Apply the RX matched RRC filter."""
350 351     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
351 352

```

```

352 353
353 354 # --- timing recovery -----
354 355
355 356 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
356 357     """Block-wise Oerder-Meyr timing recovery; returns `out` samples/symbol
357 358     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
358 359     Per block the spectral-line phase gives the offset; unwrapped across blocks
359 360     to track clock drift, then interpolated per symbol."""
360 361     n = sps
361 362     nsym = x.size // n
362 363     if nsym < 1:
363 364         return x[:0].astype(np.complex128)
364 365     xt = x[:nsym * n]
365 366     p = np.abs(xt) ** 2
366 367     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
367 368     bs = block * n
368 369     if xt.size <= bs: # single block: one global fractional offset
369 370         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
370 371     else:
371 372         nblk = xt.size // bs
372 373         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
373 374         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
374 375         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
375 376         pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
376 377         idx = np.arange(xt.size)
377 378         return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
378 379
379 380 # --- decision-directed carrier sync -----
380 381
381 382 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarra
382 383     y:
383 384         """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
384 385         Costas loop, which jitters QAM at the correct points). Sequential feedback (each
385 386         phase update depends on the prior decision) -> numba kernel when available."""
386 387         pts = ideal_points(mod)
387 388         z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
388 389         return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
389 390                        np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```

signus/pipeline.py +1 -1 · 코드북 36쪽 · 새로 쓸 줄 113

```

1 1 """End-to-end blind demodulation. Two families:
2 2     linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3 3                         classify -> fine sync -> [equalizer rescue] -> quality
4 4     fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5 5 Classification runs BEFORE fine sync so the decision-directed loop knows the
6 6 constellation. Differential demapping is an option, not a detection: dqpsk and
7 7 qpsk share a constellation."""
8 8
9 9 import json
10 10 from dataclasses import dataclass, field
11 11
12 12 import numpy as np
13 13
14 14 from . import classify as cl
15 15 from . import dsp, triage
16 16 from .channelize import extract
17 17 from .chirp import analyze_chirp, is_chirp, sweeps_band
18 18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 19 from .detect import Detection, detect

```



```

20 20 from .eq import equalize, equalize_fse
21 21 from .fsk import analyze_fsk, fsk_gate
22 22 from .sigio import Meta, parse_name, read
23 23 from .spectrum import spectrum, waterfall
24 24 from .sync import find_preamble
25 25
26 26 _BITS_CAP = 65536
27 27 _EQ_LOCK = 60.0 # below this a linear signal is worth an equalizer attempt
28 28 _EQ_GAIN = 3.0 # ...keep it only if lock improves by this much
29 29 _EQ_FLOOR = 50.0 # ...and clears this floor, so a wrong-mod fit is never locked in
30 30 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
    l, l an
31 31 # drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
    l, l , a
32 32 # confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
    l, l 's
33 33 # eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
    l, l al.
34 34 _BAUD_GAIN = 5.0 # an in-band baud hypothesis must beat the global one by this lock
35 35 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
36 36 _SYNC_LOCK = 60.0 # below this, try a repeated-preamble carrier/timing sync (packetized bursts
    l, l )
37 37 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous
    l, l .
38 38 # This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
    l, l lf
39 39 # aliases and validates a rotation at ~80 lock, that rotation wins by a thi
    l, l n
40 40 # margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
    l, l s it,
41 41 # the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
    l, l s.
42 42 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap
    l, l . A
43 43 # 65 floor was tried (to recover more moderate bursts) but an independent
44 44 # snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
    l, l 8 band
45 45 # (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
    l, l s a
46 46 # a few hard-corner rotations (a separate pre-existing gate limit): hold a
    l, l t 78.
47 47 _SYNC_ADIST = 1.5 # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 48 # soft backstop to the margin: it catches the one rotation the margin misse
    l, l s,
49 49 # but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
50 50 # large-offset recoveries. The three together give 0 confident-wrong across
51 51 # 1920 hard-corner bursts at the max recall this gate reaches.
52 52 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
53 53
54 54
55 55 @dataclass
56 56 class Result:
57 57     meta: Meta
58 58     family: str # 'linear' | 'fsk'
59 59     burst: tuple[int, int]
60 60     fc: float
61 61     baud: float
62 62     mod: str
63 63     lock: float
64 64     symbols: np.ndarray # aligned symbols (linear) / normalized levels (fsk)

```

```

65 65 bits: np.ndarray
66 66 iq_corr: np.ndarray # exportable corrected baseband
67 67 burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
68 68 symmetry: int = 0
69 69 carrier_ambiguous: bool = False
70 70 baud_conf: float = 0.0
71 71 rolloff: float | None = None
72 72 h: float | None = None # FSK modulation index
73 73 evm: float = float("nan")
74 74 mer_db: float = float("nan")
75 75 occupied: int = 0
76 76 snr_est: float = float("nan")
77 77 eq_applied: bool = False
78 78 eq_mode: str | None = None # 'sym' | 'fse'
79 79 alias_resolved: bool = False
80 80 baud_fallback: bool = False
81 81 bursts: list = field(default_factory=list)
82 82 burst_idx: int = 0
83 83 preamble: object = None # sync.Preamble when a repeated preamble drove the decode
84 84
85 85 def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86 86     z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
87 87     if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88 88         z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89 89     det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90 90           "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91 91           "baud_conf": round(self.baud_conf, 1),
92 92           "alias_resolved": self.alias_resolved,
93 93           "baud_fallback": self.baud_fallback}
94 94     det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95 95     rf = self.meta.rf_center # real RF = capture centre + baseband fc
96 96     det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97 97     if self.h is not None:
98 98         det["h"] = round(self.h, 3)
99 99     if self.preamble is not None: # a repeated preamble drove the sync
100 100         p = self.preamble
101 101         det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102 102                           "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103 103     doc = {
104 104         "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype, # dtype
105 105         "rf_center": self.meta.rf_center, # 보고에 필수: lock 0 의 1순위 용의자다
106 106
107 107         "family": self.family, "n_samples": int(self.iq_corr.size),
108 108         "burst": {"start": self.burst[0], "end": self.burst[1]},
109 109         "bursts": [{"start": s, "end": e} for s, e in self.bursts],
110 110         "burst_idx": self.burst_idx,
111 111         "detected": det,
112 112         "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
113 113                    "evm": _r(self.evm, 4), "occupied": int(self.occupied)},
114 114         "snr_est_db": _r(self.snr_est),
115 115         "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
116 116         "constellation": {"i": np.round(z.real, 4).tolist(),
117 117                           "q": np.round(z.imag, 4).tolist()},
118 118         "bits": "".join(map(str, self.bits[:_BITS_CAP])),
119 119     }
120 120     if views and self.burst_x is not None:
121 121         doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
122 122         doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
123 123     return doc

```

```

124 124
125 125     def save_iq(self, path: str) -> None:
126 126         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
127 127
128 128     def save_symbols(self, path: str) -> None:
129 129         np.save(path, self.symbols.astype(np.complex64))
130 130
131 131     def save_bits(self, path: str, packed: bool = False) -> None:
132 132         if packed:
133 133             np.packbits(self.bits).tofile(path)
134 134         else:
135 135             with open(path, "w") as fh:
136 136                 fh.write("".join(map(str, self.bits)))
137 137
138 138     def save_report(self, path: str) -> None:
139 139         with open(path, "w") as fh:
140 140             json.dump(self.to_json(), fh, indent=1)
141 141
142 142
143 143     def _r(v: float, nd: int = 2) -> float | None:
144 144         return None if v is None or not np.isfinite(v) else round(float(v), nd)
145 145
146 146
147 147     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
148 148         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
149 149         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
150 150
151 151
152 152     def _blocks(n: int) -> int:
153 153         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
154 154         low SNR) wants fewer, longer segments; only long records need more for
155 155         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
156 156         return int(np.clip(n // 30000, 1, 8))
157 157
158 158
159 159     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
160 160         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
161 161         alpha = dsp.est_rolloff(xd, fs, baud)
162 162         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
163 163         raw = dsp.timing(ym, 4)
164 164         mod = cl.classify(raw, symmetry)
165 165         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
166 166         # the bulk phase first kills the loop transient that would corrupt the first symbols
167 167         syms = _fine(raw, mod)
168 168         return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
169 169
170 170
171 171     def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
172 172         """Equalize with a seed constellation, then let the OPENED eye pick the mod:
173 173         heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
174 174         with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
175 175         z1 = eq_fn(z, seed_mod)
176 176         cand_s = []
177 177         for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstrin
178 178             g
179 179             s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
180 180             cand_s.append((cl.quality(s, m), s, m))
181 181         best = max(cand_s, key=lambda c: c[0].lock)
182 182         # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
183 183         # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).

```

```

183 183 # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
184 184 # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
185 185 # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
186 186 lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
187 187 if lo and best[2] == "qpsk":
188 188     return lo[0]
189 189 return best
190 190
191 191
192 192 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
193 193     burst: int | None = None) -> Result:
194 194     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
195 195     `burst` selects one detected burst (default: the most energetic)."""
196 196     if x.size:
197 197         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
198 198         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
199 199         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
200 200         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
201 201         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
202 202         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
203 203         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
204 204         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
205 205         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
206 206             x = x / scale
207 207     x = dsp.analytic(x)
208 208     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
209 209     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
210 210         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
211 211     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
212 212     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-var
213 213         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
214 214         er/
215 215         # underflow) stay invariant. Normal captures rms~0(1
216 216         # untouched -> the acceptance sweep is byte-identical
217 217         .
218 218     bursts = dsp.find_bursts(x, meta.fs)
219 219     if burst is None or not 0 <= burst < len(bursts):
220 220         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
221 221     s, e = bursts[burst]
222 222     xb = x[s:e] if e - s >= 64 else x
223 223
224 224     # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
225 225     # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
226 226     # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
227 227     # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
228 228     if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
229 229         info = analyze_chirp(xb, meta.fs)
230 230         kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
231 231         raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) - "
232 232             f"성상도 복조 대상이 아닙니다 (서베이 모드에서 특성 보고)")
233 233
234 234     if fsk_gate(xb, meta.fs):
235 235         r = analyze_fsk(xb, meta.fs)
236 236         sym = np.asarray(r["symbols"], dtype=np.complex128)
237 237         return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
238 238             sym, r["bits"], xb, burst_x=xb, h=r["h"],

```

```

239 239 fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
240 240 fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
241 241 amb = abs(symmetry * fc) > 0.4 * meta.fs
242 242 xd = dsp.mix(xb, meta.fs, fc)
243 243
244 244 baud, conf = dsp.est_baud(xd, meta.fs)
245 245 fell = False
246 246 alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
247 247 bw = dsp.occupied_bw(xd, meta.fs)
248 248 if bw and not (0.72 * bw <= baud <= 1.05 * bw):
249 249     # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
250 250     # (and under harsh channels) data junk outruns the true line. Demod the
251 251     # in-band hypotheses too and let lock decide, with a clear margin.
252 252     for lo in (0.80, 0.67):
253 253         b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
254 254         if abs(b2 - baud) <= 0.01 * b2:
255 255             continue
256 256         t = _demod(xd, meta.fs, b2, symmetry)
257 257         if t[5].lock > q.lock + _BAUD_GAIN:
258 258             alpha, ym, raw, mod, syms, q = t
259 259             baud, conf, fell = b2, c2, True
260 260
261 261 if q.lock < _SHORT_LOCK:
262 262     # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
263 263     # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
264 264     # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
265 265     # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
266 266     for nb in (2, 1):
267 267         b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
268 268         if abs(b2 - baud) <= 0.01 * b2:
269 269             continue
270 270         t = _demod(xd, meta.fs, b2, symmetry)
271 271         if t[5].lock > q.lock + _BAUD_GAIN:
272 272             alpha, ym, raw, mod, syms, q = t
273 273             baud, conf, fell = b2, c2, True
274 274
275 275 eq_applied, eq_mode, pre = False, None, None
276 276 if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
277 277     # CMA is modulus-only, so it opens the eye without knowing the constellation.
278 278     qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
279 279     mode = "sym"
280 280     if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
281 281         # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
282 282         # the clock-tracked 2 sps stream (unaliaised spectrum, longer inverse)
283 283         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
284 284             dsp.timing(ym, 4, out=2), mod, symmetry)
285 285         if q2.lock > qe.lock:
286 286             qe, se, me, mode = q2, s2, m2, "fse"
287 287         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
288 288         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
289 289             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
290 290     elif mod in ("16qam", "32qam", "64qam"):
291 291         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
292 292         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
293 293         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
294 294         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
295 295         .
296 296         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
297 297         mode = "sym"
298 298         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:

```

```

298 298 # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
299 299 # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
300 300 # QAM (no lower order) never pays for the FSE.
301 301 qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
302 302                 dsp.timing(ym, 4, out=2), mod, symmetry)
303 303 mode = "fse"
304 304 if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
305 305     syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
306 306
307 307 if q.lock < _SYNC_LOCK:
308 308     # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
309 309     # carrier/ baud/ timing from only a few symbols, where the blind M-th-power estimator (nee
310 310     # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
311 311     # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
312 312     # reaches here. Detect the preamble, mix by its CF0, demod the DATA past it, keep-best -
313 313     # random / no-preamble signal yields no plateau or a CF0 that does not improve lock an
314 314     # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
315 315     # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
316 316     # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
317 317     ps = find_preamble(xb, meta.fs, baud_hint=baud)
318 318     if ps is not None:
319 319         # The Schmidl-Cox CF0 is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
320 320         # rotates a whole constellation position per symbol, so the eye still looks locked b
321 321         # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
322 322         # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
323 323         # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
324 324         # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
325 325         step = meta.fs / ps.period
326 326         k0 = round((fc - ps.cfo_hz) / step)
327 327         cands = []
328 328         for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
329 329             for k in range(k0 - 3, k0 + 4):
330 330                 cfo = ps.cfo_hz + k * step
331 331                 data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
332 332                 if data.size < 256:
333 333                     continue
334 334                 for sm in (2, 4, 8):
335 335                     t = _demod(data, meta.fs, float(b2), sm)
336 336                     cands.append((t[5].lock, cfo, float(b2), sm, t))
337 337                 cands.sort(key=lambda c: -c[0])
338 338         # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
339 339         # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
340 340         # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
341 341         # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
342 342         # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -

```

```

    - a
343 343 # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alone
    - ne
344 344 # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
    - . Two
345 345 # conjunctive conditions favour precision over recall: (1) a strong absolute lock, and
    - nd
346 346 # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
    - a
347 347 # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
348 348 # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); on the
    - n the
349 349 # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
350 350 if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
351 351     top = cands[0]
352 352     other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
353 353     anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
354 354     if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
355 355         alpha, ym, raw, mod, syms, q = top[4]
356 356         fc, baud, symmetry = top[1], top[2], top[3]
357 357         eq_applied, eq_mode, pre = False, None, ps
358 358         # diagnostics must describe the ACCEPTED decode, not the abandoned blind
359 359         # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
360 360         # keeps alias_resolved False), the baud from the preamble period (no
361 361         # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
362 362         fc0, conf, fell = fc, 0.0, False
363 363         amb = abs(symmetry * fc) > 0.4 * meta.fs
364 364
365 365 bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
366 366     else demap_bits(syms, mod)
367 367 return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
368 368     burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
369 369     rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
370 370     snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
371 371     alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
372 372     bursts=bursts, burst_idx=burst, preamble=pre)
373 373
374 374
375 375 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
376 376     dtype: str | None = None, endian: str | None = None,
377 377     bitrev: bool | None = None, diff: bool = False,
378 378     burst: int | None = None, rf: float | None = None) -> Result:
379 379     """Analyze a file; explicit args override SigMF/filename tokens."""
380 380     from .sigio import parse_sigmf
381 381     m = parse_sigmf(path) or parse_name(path)
382 382     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
383 383         endian or m.endian, m.bitrev if bitrev is None else bitrev,
384 384         rf_center=rf if rf is not None else m.rf_center)
385 385     x, meta = read(path, meta)
386 386     return analyze(x, meta, diff, burst)
387 387
388 388
389 389 # --- wideband survey: many emitters in one capture -----
390 390
391 391 @dataclass
392 392 class Emitter:
393 393     detection: Detection
394 394     kind: str # linear|fsk|chirp|analog|tone|tooshort|error
395 395     abs_fc: float # emitter carrier vs capture centre (Hz)
396 396     result: Result | None = None # demod result for digital kinds

```

```

397 397 info: dict | None = None          # characterization for non-digital kinds (e.g. chirp params)
398 398
399 399 def to_json(self) -> dict:
400 400     d = self.detection
401 401     doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
402 402           "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
403 403                "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
404 404                "baud_hint": round(d.baud_hint, 1)}}
405 405     if self.info is not None:
406 406         doc["info"] = self.info
407 407     if self.result is not None:
408 408         r = self.result
409 409         doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
410 410                  mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
411 411     return doc
412 412
413 413 def to_detail(self, rf_center: float | None = None) -> dict:
414 414     """Web drill-down payload: the box summary plus, for a digital emitter, the
415 415     full analyze result so the UI reuses its single-signal detail view verbatim.
416 416     The channel is demodulated at baseband (no rf_center), and r.fc is only the
417 417     residual within-channel offset -- so the emitter's absolute RF is the capture
418 418     centre plus abs_fc (the full offset from capture centre), injected here."""
419 419     doc = self.to_json()
420 420     if self.result is not None:
421 421         doc["result"] = self.result.to_json()
422 422         if rf_center is not None:
423 423             doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
424 424     return doc
425 425
426 426
427 427 @dataclass
428 428 class Survey:
429 429     meta: Meta
430 430     emitters: list[Emitter] = field(default_factory=list)
431 431
432 432     def to_json(self) -> dict:
433 433         return {"fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,
434 434               "n_emitters": len(self.emitters),
435 435               "emitters": [e.to_json() for e in self.emitters]}
436 436
437 437
438 438 def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
439 439     """Detect every emitter in a wideband capture, extract each to its own baseband
440 440     channel, and demodulate the digital ones with the unchanged single-signal chain.
441 441     A capture holding one signal that fills the band collapses to a single emitter."""
442 442     xa = dsp.analytic(x)
443 443     xa = xa - xa.mean()
444 444     emitters = []
445 445     for d in detect(xa, meta.fs, **detect_kw):
446 446         ch, fs_ch = extract(xa, meta.fs, d)
447 447         if ch.size < 256:
448 448             emitters.append(Emitter(d, "tooshort", d.fc))
449 449             continue
450 450         kind = triage.family(ch, fs_ch)
451 451         if kind in ("linear", "fsk"):
452 452             # triage only gates digital vs not; analyze's own family is authoritative.
453 453             # Isolate each channel: a degenerate one must not abort the whole survey.
454 454             try:
455 455                 r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
456 456                 emitters.append(Emitter(d, r.family, d.fc + r.fc, r))

```



```

457 457         except Exception:
458 458             emitters.append(Emitter(d, "error", d.fc))
459 459     else:
460 460         info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
461 461         emitters.append(Emitter(d, kind, d.fc, info=info))
462 462     return Survey(meta, emitters)
463 463
464 464
465 465 def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
466 466     """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
467 467     is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
468 468     result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
469 469     capture overview waterfall and every emitter's drill-down detail. Additive: does
470 470     not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
471 471     xa = dsp.analytic(x)
472 472     xa = xa - xa.mean()
473 473     if len(detect(xa, meta.fs)) <= 1:
474 474         r = analyze(x, meta, diff=diff)
475 475         out = {"mode": "single", "result": r.to_json()}
476 476         if len(r.bursts) > 1: # multi-burst signal -> whole-record map for burst selection
477 477             out["overview"] = {"n": int(xa.size), "fs": meta.fs,
478 478                               "waterfall": waterfall(xa, meta.fs)}
479 479         return out
480 480     sv = survey(x, meta, diff=diff)
481 481     return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
482 482           "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),
483 483                       "waterfall": waterfall(xa, meta.fs)},
484 484           "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}
485 485
486 486
487 487 def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
488 488               dtype: str | None = None, endian: str | None = None,
489 489               bitrev: bool | None = None, diff: bool = False,
490 490               rf: float | None = None) -> Survey:
491 491     """Survey a capture file; explicit args override SigMF/filename tokens."""
492 492     from .sigio import parse_sigmf
493 493     m = parse_sigmf(path) or parse_name(path)
494 494     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
495 495               endian or m.endian, m.bitrev if bitrev is None else bitrev,
496 496               rf_center=rf if rf is not None else m.rf_center)
497 497     x, meta = read(path, meta)
498 498     return survey(x, meta, diff=diff)

```